

CAIRO UNIVERSITY

FACULTY OF COMPUTING AND ARTIFICIAL INTELLIGENCE



NETWORKS AND CYBER SECURITY PROGRAM

Under the Supervision of

Prof. Iman Sanad

Graduation Project Documentation

"CATF-IDS"

Context-Adaptive Threshold Fusion
Intrusion Detection System for Industrial IoT

Submitted by:

Abdlrahman Amr
Ahmed Mamdouh
Seifeldein Sayed
Mohamed Othman

20226169
20225002
20225019
20225037

Abstract (English)

Industrial Internet of Things (IIoT) environments present a unique and underexplored security challenge: cyberattacks may simultaneously manifest across cyber and physical data planes, yet every published intrusion detection system (IDS) for the widely used TON_IoT benchmark dataset analyzes network traffic alone and ignores physical sensor readings, system logs, and endpoint telemetry entirely.

This project presents CATF-IDS (Context-Adaptive Threshold Fusion IDS), a nine-layer governance architecture that simultaneously fuses four heterogeneous IIoT data streams — network traffic (27 features), IoT physical sensor readings (6 features), Linux system logs (10 features), and Windows telemetry (5 features) — to produce a unified per-event threat decision with full LIME-based explainability.

The system implements an adaptive detection threshold driven by EWMA score-distribution drift, a contextual per-event threshold adjustment using tanh-saturated multi-factor algebra, an Energy Finite State Machine (FSM) for temporal multi-stage campaign detection, and a three-tier behavioral memory system combining DBSCAN clustering, campaign progression matching, and a confidence-gated online learner. A FixMatch-style consistency-regularized pseudo-label self-training mechanism enables Phase 2 unlabeled adaptation during deployment without any human label access.

Evaluated under strict five-fold cross-validation on the balanced TON_IoT dataset, CATF-IDS achieves $\text{ROC-AUC} = 0.922 \pm 0.013$, $\text{F1-Score} = 0.827$, $\text{Precision} = 0.944$, and $\text{FPR} = 0.111$ under scientific balanced conditions. Phase 2 blindfolded adaptation further reduces FPR to 0.159 while simultaneously improving all five evaluation metrics without label access. An ablation study confirms that the FSM escalation mechanism accounts for 73% of the deployment FPR, documenting the explicit cost of temporal campaign detection capability that no published single-modality system provides.

ملخص (عربي)

وهو بنية حوكمة من تسع طبقات مصممة خصيصاً للكشف عن التسلل في بيانات CATF-IDS يُقدّم هذا المشروع نظام يدمج النظام أربعة تدفقات بيانات غير متجانسة في آن واحد: حركة الشبكة، وقراءات (IIoT). إنترنت الأشياء الصناعي لإنتاج قرار تهديد شامل، وبيانات قياس Windows، والمستشعرات الفيزيائية لأجهزة إنترنت الأشياء، وسجلات نظام LIME لكل حدث مع تفسير كامل قائم على خوارزمية.

وآلة حالة (EWMA) يعتمد النظام على عتبة كشف تكيفية مدفوعة بانجراف توزيع الدرجات عبر المتوسط المتحرك الأسّي للكشف عن حملات الهجوم متعددة المراحل عبر الزمن، ونظام ذاكرة سلوكي ثلاثي (Energy FSM) نهائية للطاقة ومطابقة تسلسلات التهديد وآلية تعلم عبر الإنترنت. تُحقق المرحلة الثانية التكيف DBSCAN المستويات يجمع بين تجميع دون أي وصول إلى تسميات بشرية FixMatch غير المُشرف خلال النشر باستخدام التسميات الزائفة المُرشحة باتساق تحت التحقق التقاطعي الصارم من خمسة أطواء أن النظام يُحقق TON_IoT تُظهر التجارب المُجرّاة على مجموعة بيانات FPR ودقة = 0.944، ومعدل إيجابية كاذبة، $\text{F1} = 0.827$ وقيمة $\text{ROC-AUC} = 0.922 \pm 0.013$ مساحة تحت المنحنى في ظل ظروف التقييم المتوازن. وتُوضح دراسة الاستئصال أن آلية التصعيد بمكبنة الحالة المحدودة تُسهم بنسبة 73% من معدل الإيجابية الكاذبة في النشر الحقيقي، مما يُوثّق التكلفة الصريحة لقدرة الكشف عن الحملات الزمنية التي لا 73% يوفرها أي نظام منشور أحادي الطريقة.

Table of Contents

List of Abbreviations

AUC: Area Under the ROC Curve

CATF-IDS: Context-Adaptive Threshold Fusion Intrusion Detection System

C2: Command and Control

CV: Cross-Validation

DBSCAN: Density-Based Spatial Clustering of Applications with Noise

DoS: Denial of Service

EWMA: Exponentially Weighted Moving Average

FPR: False Positive Rate

FSM: Finite State Machine

IDS: Intrusion Detection System

IIoT: Industrial Internet of Things

IoT: Internet of Things

LIME: Local Interpretable Model-Agnostic Explanations

LR: Logistic Regression

ML: Machine Learning

RF: Random Forest

ROC: Receiver Operating Characteristic

SOC: Security Operations Center

TON_IoT: Things Of Network IoT dataset (UNSW)

UNSW: University of New South Wales

Chapter 1: Introduction

1.1 Background and Motivation

The convergence of traditional operational technology with Internet Protocol networks has created the Industrial Internet of Things (IIoT) — a vast ecosystem of smart sensors, actuators, programmable logic controllers, and edge computing devices embedded in critical infrastructure including energy grids, water treatment facilities, smart manufacturing plants, and logistics networks. Unlike conventional enterprise computing environments, IIoT systems exercise physical processes: a compromised thermostat can cause dangerous temperature conditions; a falsified GPS reading can misdirect autonomous vehicles; a manipulated Modbus controller can disable industrial machinery.

This cyber-physical convergence fundamentally changes the threat model for intrusion detection. An attacker targeting an IIoT environment need not steal data in the conventional sense — disrupting a physical process or causing a sensor to report false readings can be equally or more damaging. The National Institute of Standards and Technology (NIST) and the European Union Agency for Cybersecurity (ENISA) both identify this broadened attack surface as one of the defining security challenges of the current decade.

Existing intrusion detection systems, however, have not kept pace with this expanded threat model. The overwhelming majority of published IDS research focuses exclusively on network traffic analysis. While network-layer anomalies are important indicators of attack, they represent only one dimension of a multi-dimensional threat landscape. An attacker who maintains network traffic within expected parameters while manipulating physical sensor readings — a technique increasingly documented in targeted industrial control system attacks — is invisible to any network-only IDS, regardless of its algorithmic sophistication.

The TON_IoT dataset, published by the University of New South Wales and widely adopted as the standard IIoT security benchmark, uniquely provides co-temporal data from four distinct planes: network packet flows, IoT physical device telemetry, Linux system logs, and Windows endpoint monitoring. Despite this multi-modal richness, every published system evaluated on this dataset uses only the network traffic component. CATF-IDS is designed to address this gap.

1.2 Problem Definition

The problem addressed by this project can be stated precisely: no existing intrusion detection system for IIoT environments simultaneously addresses all four of the following documented deficiencies:

- Single-modality limitation: published systems analyze network traffic exclusively, ignoring physical sensor anomalies, host-level system telemetry, and endpoint indicators of compromise that are visible in the same IIoT deployment environments.
- Temporal blindness: per-event classifiers evaluate each network event in isolation. Multi-stage attack campaigns — where reconnaissance, initial access, lateral movement, and persistence occur as a progression of individually borderline events — are entirely undetectable by memoryless classifiers.
- Training-deployment distribution mismatch: systems trained on balanced data are deployed into environments where attack prevalence differs substantially from training conditions. A static detection threshold calibrated to a 50/50 training distribution will be systematically miscalibrated during a sustained attack period, but no published system provides a principled mechanism for online threshold adaptation.
- Evaluation dishonesty: datasets such as TON_IoT contain 80–95% attack-labeled events. Systems reporting 97–99% accuracy on these datasets are exploiting class imbalance rather than demonstrating genuine discriminative learning. False positive rate, the operationally critical metric, is rarely reported.

CATF-IDS addresses all four deficiencies simultaneously through a nine-layer governance architecture that fuses four heterogeneous data modalities, maintains temporal state across event sequences, adapts its detection threshold to observed deployment conditions, and provides honest evaluation under both controlled and realistic deployment distributions.

1.3 Project Objectives and Proposed Solution

The primary objectives of this project are:

- Design and implement a multi-modal IDS that fuses network traffic, IoT physical sensor readings, Linux system logs, and Windows telemetry through a principled evidence combination framework.
- Separate detection (whether an individual event is suspicious) from escalation (what the system-level threat level should be), preventing the positive-feedback alert fatigue loop characteristic of conventional IDS systems.
- Implement adaptive threshold governance that tracks deployment score-distribution drift and adjusts the detection boundary without requiring labeled deployment data.
- Provide temporal threat intelligence through an Energy Finite State Machine that connects individual events into campaign-level narratives, detecting multi-stage attacks across extended event sequences.
- Validate all results through five-fold stateful cross-validation with confidence intervals, and provide honest dual FPR reporting that distinguishes scientific controlled evaluation from realistic deployment conditions.
- Enable Phase 2 unlabeled deployment adaptation through consistency-regularized pseudo-label self-training, improving all evaluation metrics without human label access.

The proposed solution, CATF-IDS (Context-Adaptive Threshold Fusion IDS), implements these objectives through a nine-layer architecture described in detail in Chapters 4 and 5.

1.4 Project Scope and Limitations

CATF-IDS is evaluated on the TON_IoT benchmark dataset (University of New South Wales), which provides the richest publicly available multi-modal IIoT security dataset. The system is

designed for offline batch evaluation and demonstration, with the REST API providing a pathway to real-time deployment. The following limitations are explicitly acknowledged:

- Dataset density: TON_IoT's deployment distribution of 82% attacks, generated under controlled laboratory conditions, significantly differs from real operational IIoT environments where attacks constitute 5–10% of traffic. This density prevents the confident normal tracker from accumulating sufficient samples for baseline recalibration in Phase 2, though pseudo-label self-training succeeds regardless.
- Cross-dataset validation: the four-modality fusion architecture cannot be evaluated on alternative datasets (Bot-IoT, CICIDS-2017, WUSTL-IIoT) because none provide co-temporal data from all four required data planes. Network-only evaluation on these datasets would require disabling three of four modality channels, producing results incomparable to the full system.
- Physical device integration: the system processes telemetry from physical IoT devices via the TON_IoT dataset but does not integrate with live physical devices in the current implementation.
- Scalability: the current implementation processes events as a streaming pipeline over a fixed dataset. Distributed deployment across multiple network segments and sensor networks would require additional engineering.

1.5 Development Methodology

CATF-IDS was developed following an iterative research methodology structured around two major phases:

Phase 1 — Supervised Learning and Governance Architecture

Phase 1 implemented the complete nine-layer architecture, trained all component models on labeled balanced data, and evaluated the system under five-fold cross-validation with comprehensive metric aggregation. The key engineering iterations within Phase 1 were:

- Initial implementation of the three-modality RF layer models and LR fusion classifier.
- Addition of the policy engine and adaptive threshold components, with empirical threshold calibration via F-beta grid search.
- Implementation of the Energy FSM and behavioral memory system with DBSCAN clustering and campaign detection.
- Discovery and resolution of the FSM-FPR tension through ablation study and addition of the confidence gate mechanism.
- Five-fold stateful cross-validation with four-level parameter aggregation (ensemble weighting, threshold selection, trust initialization, FSM calibration).

Phase 2 — Unlabeled Deployment Adaptation

Phase 2 added the blindfolded adaptation mechanism, which operates entirely without label access during the inference stream. The primary innovation is FixMatch-style consistency-regularized pseudo-label self-training with adaptive confidence thresholding tied to the current detection boundary, rather than a static absolute threshold.

1.6 Tools and Technologies Used

The following tools and technologies were used in the implementation:

Tool / Technology	Category	Role in CATF-IDS
Python 3.x	Core Language	Entire system implemented as ids_complete.py (5,651 lines)
scikit-learn	ML Library	Random Forest, Logistic Regression, DBSCAN, StandardScaler
pandas	Data Processing	Temporal multi-modal fusion via merge_asof (causal alignment)
NumPy	Numerical Computing	Behavioral physics feature engineering (27 features)
LIME	Explainability	Per-event plain-English feature attribution for SOC analysts
TON_IoT Dataset (UNSW)	Evaluation Benchmark	38 CSV files across 4 modalities (Net+IoT+Log+Win)
FastAPI	API Framework	REST API for real-time inference and system status monitoring
matplotlib	Visualization	Evaluation plots, FSM distribution, campaign timelines

Table 1.1 — Tools and Technologies Used in CATF-IDS

1.7 Project Timeline

The project was executed over two semesters following the structured phases outlined below:

Phase	Milestone	Duration	Status
Phase 1-A	Data loading, preprocessing, temporal fusion pipeline	3 weeks	Complete
Phase 1-B	RF layer models + LR fusion + policy engine	3 weeks	Complete
Phase 1-C	Adaptive threshold + contextual threshold engine	2 weeks	Complete
Phase 1-D	Energy FSM + behavioral memory + online learner	3 weeks	Complete
Phase 1-E	5-fold stateful CV + 4-level aggregation + ablation study	2 weeks	Complete
Phase 1-F	Confidence gate implementation and FPR reduction	1 week	Complete
Phase 2	Blindfolded adaptation: FixMatch consistency + pseudo-label retraining	2 weeks	Complete
REST API	FastAPI implementation for real-time inference	1 week	Complete
Documentation	Final report, poster, presentation	1 week	In Progress

Table 1.2 — Project Timeline

1.8 Report Organization

This report is organized as follows. Chapter 2 surveys related work in IIoT intrusion detection and establishes the gap that CATF-IDS addresses. Chapter 3 presents the system requirements analysis including stakeholder analysis, functional and non-functional requirements, and use case descriptions. Chapter 4 describes the research methodology including the dataset, preprocessing pipeline, feature engineering, and evaluation design. Chapter 5 details the implementation of all nine layers. Chapter 6 presents the comprehensive experimental evaluation including cross-validation results, the ablation study, Phase 2 blindfolded adaptation results, and comparison with published literature. Chapter 7 summarizes the contributions and directions for future work.

Chapter 2: Related Work

2.1 Overview of Existing IDS Systems for IIoT

Intrusion detection for Industrial IoT environments has attracted substantial research attention following high-profile incidents including Stuxnet, the Ukrainian power grid attack, and the

Triton/TRISIS attack on safety instrumented systems. The research landscape, however, reveals a persistent gap between the theoretical requirements of IIoT security and the actual implementations evaluated in the academic literature.

The TON_IoT dataset, published by Alsaedi et al. (2020) at the University of New South Wales, has become the dominant benchmark for IIoT IDS evaluation. It provides network packet logs alongside IoT physical telemetry, Linux system process and resource logs, and Windows endpoint monitoring — making it the only publicly available benchmark with true multi-modal co-temporal data. Despite this richness, virtually all published systems using this dataset extract only the network traffic component.

2.1.1 Machine Learning Approaches

The dominant paradigm in IIoT IDS research applies supervised machine learning classifiers to network traffic features. Alsaedi et al. (2020) demonstrated a Random Forest baseline achieving approximately 97% accuracy on the network component of TON_IoT, establishing a performance reference that subsequent systems have consistently matched or exceeded on the same data. However, as noted in Section 1.2, accuracy on an 82% attack-labeled dataset provides limited information about a classifier's genuine discriminative ability.

Recent work has investigated ensemble methods and gradient boosting for IDS. Systems combining Random Forest, XGBoost, LightGBM, and CatBoost have reported accuracy values of 91–99% on network TON_IoT data. Moustafa and Turnbull (2023) proposed a stacked generalization approach achieving 90% accuracy with improved stability, while noting the class imbalance limitations that affect all such comparisons.

2.1.2 Deep Learning Approaches

Several recent papers have applied recurrent and transformer-based architectures to IDS. GRU, LSTM, Bi-LSTM, and attention-based models have reported 97–99% accuracy on TON_IoT network data. ROAST-IoT (2023) proposed a CNN-attention hybrid achieving 99.78% accuracy with ROC-AUC ≈ 0.998 — values that, as argued in Section 6.4 of this report, reflect the near-trivial nature of classification on heavily imbalanced data rather than genuine performance at the stated level.

A masked autoencoder approach (MAE + LightGBM) was proposed for semi-supervised IDS with limited labeled data, achieving 99/93/95% precision/recall/F1 on TON_IoT. While this is among the more sophisticated recent systems, it remains network-only and does not address the temporal reasoning limitation.

2.1.3 Graph-Based and Self-Supervised Approaches

Graph Neural Network (GNN) based self-supervised learning has been proposed for IoT traffic differentiation without explicit labels. These approaches learn structural representations of network traffic patterns that generalize to novel attack types. SSTD and related systems demonstrate promising generalization, but remain fundamentally single-modality.

2.2 Comparative Analysis: What Distinguishes CATF-IDS

Table 2.1 presents a systematic comparison of CATF-IDS against the most relevant published systems on the TON_IoT benchmark:

System / Paper	Modalities	Temporal	FPR Reported	CV Validated
Alsaedi et al. 2020 (TON_IoT baseline)	Network	No	No	No
GRU/LSTM/Bi-LSTM (2024)	Network	No	No	No
ROAST-IoT CNN-Attention (2023)	Network	No	No	No
Stacked Ensemble AutoML (2024)	Network	No	No	No
MAE + LightGBM Semi-supervised	Network	No	No	No
CATF-IDS (This Work)	Net+IoT+Log+WinFSM + Campaign		Both (0.111/0.196)	5-fold $\pm$ CI

Table 2.1 — Comparison of CATF-IDS with Published TON_IoT Systems

The comparison reveals four dimensions in which CATF-IDS is distinguished from all published systems:

- Multi-modality: CATF-IDS is the only system to fuse all four available data planes from TON_IoT. Every other system uses network data exclusively.
- Temporal reasoning: CATF-IDS is the only system with explicit temporal campaign detection. The Energy FSM connects individual events into campaign narratives across sequences of 50+ events.
- Honest evaluation: CATF-IDS reports FPR under both balanced scientific conditions (FPR = 0.111) and realistic deployment conditions (FPR = 0.196), documents the gap, and explains its mechanism through a controlled ablation study. No published system provides this level of evaluation transparency.
- Validated stability: CATF-IDS provides confidence-interval validated AUC (0.922 ± 0.013) across five independent cross-validation folds. No comparable stability evidence is provided by published systems.
- Unlabeled adaptation: Phase 2 demonstrates all-metric improvement under completely label-free deployment adaptation, a capability not implemented in any published TON_IoT IDS system.

2.3 Semi-Supervised and Continual Learning Approaches

The broader IDS literature beyond TON_IoT includes several relevant semi-supervised and continual learning approaches that inform the Phase 2 design of CATF-IDS.

Self-training with pseudo-labels (Yarowsky, 1995) is the classical semi-supervised technique: a classifier trained on labeled data produces confident predictions on unlabeled data, which are then added to the training set for iterative refinement. SSID and related systems (2024) apply self-training to IDS, demonstrating that high-confidence pseudo-labels from network traffic classifiers can improve detection of novel attack variants.

FixMatch (Sohn et al., 2020) strengthens pseudo-label selection through consistency regularization: a prediction is accepted as a pseudo-label only if it remains stable under perturbation of the input. CATF-IDS Phase 2 adapts this principle to tabular streaming IDS data by applying Gaussian perturbations to the evidence vector features and accepting pseudo-labels only when the fusion classifier's prediction is stable under all K perturbations.

Continual learning systems (CITADEL, 2024) address catastrophic forgetting when classifiers are retrained on new pseudo-labeled deployment data. CATF-IDS addresses this through a replay

buffer mechanism that mixes original labeled training examples into every pseudo-label retraining batch.

Chapter 3: System Analysis

3.1 Stakeholder Analysis

CATF-IDS serves a diverse set of stakeholders with distinct needs and concerns. Understanding these stakeholder perspectives informed the system's design requirements, evaluation methodology, and output format.

Stakeholder	Role	Primary Need	Success Criterion
IIoT Security Operators	Primary users — monitor IDS output in real time	Low false alarm rate; actionable alerts	FPR < 0.20; LIME explanation per alert
SOC Analysts	Investigate and triage detected incidents	Understand why each event was flagged	Full per-event explanation with confidence
IIoT Facility Managers	Responsible for OT/IT convergence security	System reliability; campaign-level visibility	Campaign detection; FSM state summary
Cybersecurity Researchers	Evaluate multi-modal adaptive IDS designs	Reproducible results with honest metrics	CV-validated AUC with confidence intervals
Critical Infrastructure Agencies	National-level IIoT security oversight	Detect sophisticated multi-stage campaigns	Campaign detection: Recon→Access→Persist

Table 3.1 — Stakeholder Analysis

3.2 Functional Requirements

The functional requirements of CATF-IDS are organized by system capability:

FR-1: Multi-Modal Data Fusion

- FR-1.1: The system shall ingest and process network traffic, IoT physical sensor readings, Linux system logs, and Windows telemetry simultaneously.
- FR-1.2: The system shall temporally align all four data streams using causal nearest-neighbor time joining (merge_asof with direction=backward).
- FR-1.3: The system shall produce a single fused event representation for each aligned event tuple.

FR-2: Machine Learning Classification

- FR-2.1: The system shall train independent Random Forest classifiers for each data modality.
- FR-2.2: The system shall combine modality outputs into a unified fusion score through a trained meta-classifier.
- FR-2.3: The system shall calibrate modality probability outputs to correct for overconfidence bias.

FR-3: Adaptive Detection Threshold

- FR-3.1: The system shall maintain a running estimate of the deployment score distribution via EWMA.
- FR-3.2: The system shall adjust the detection threshold based on the measured drift between deployment and training score distributions.
- FR-3.3: The system shall apply per-event contextual adjustments to the threshold based on modality disagreement and trust.

FR-4: Temporal Threat Intelligence

- FR-4.1: The system shall maintain a running energy score that accumulates risk across event sequences.
- FR-4.2: The system shall transition between threat states (SAFE, SUSPICIOUS, ELEVATED, CRITICAL) based on accumulated energy.
- FR-4.3: The system shall detect multi-stage attack campaign progressions matching known templates.

FR-5: Explainability

- FR-5.1: The system shall provide LIME-based feature attribution for each detected attack event.
- FR-5.2: The system shall provide a plain-English decision narrative including the primary reason, modality agreement, risk breakdown, and behavioral intent.

FR-6: Phase 2 Adaptation

- FR-6.1: The system shall collect high-confidence pseudo-labels from the deployment stream without label access.
- FR-6.2: The system shall apply consistency regularization to filter boundary pseudo-labels.
- FR-6.3: The system shall retrain the fusion classifier periodically on accumulated pseudo-labels with replay buffer mixing.

3.3 Non-Functional Requirements

Requirement	Category	Target Value
Detection accuracy	Performance	ROC-AUC ≥ 0.90 under 5-fold cross-validation
False positive rate	Performance	FPR ≤ 0.20 under full deployment distribution
Inference latency	Performance	$\leq 2\text{ms}$ per event (excluding LIME)
Explainability	Usability	Plain-English per-event explanation for every detected attack
Model stability	Reliability	AUC standard deviation ≤ 0.02 across 5-fold CV
Unlabeled adaptation	Adaptability	Phase 2 FPR $\leq$ Phase 1 FPR without any label access
Code maintainability	Maintainability	Single self-contained Python file with full documentation
REST API response time	Performance	$\leq 100\text{ms}$ for /predict endpoint under normal load

Table 3.2 — Non-Functional Requirements

3.4 Use Case Descriptions

UC-1: Real-Time Threat Detection

Actor: IIoT Security Operator. Precondition: System trained and running. Main flow: (1) New event arrives from the fused data stream. (2) System extracts features and computes modality probabilities. (3) Fusion classifier produces unified threat score. (4) Adaptive threshold and FSM determine threat level. (5) System produces decision (NORMAL/ATTACK) with confidence. (6) If attack detected, LIME explanation is generated. (7) Result is logged and operator alerted at ELEVATED or higher. Postcondition: Decision recorded in behavioral memory.

UC-2: Campaign Investigation

Actor: SOC Analyst. Precondition: A campaign has been detected by the behavioral memory system. Main flow: (1) Analyst queries the REST API for active campaigns. (2) System returns campaign type, severity, progression sequence, and event indices. (3) Analyst requests full explanation for individual events within the campaign. (4) System returns plain-English narrative with LIME attribution. Postcondition: Analyst has sufficient information to initiate incident response.

UC-3: Phase 2 Deployment Adaptation

Actor: System (automated). Precondition: Phase 1 training complete; Phase 2 blindfolded deployment running. Main flow: (1) Every 100 high-confidence events accepted by the consistency gate are accumulated as pseudo-labels. (2) System retrains fusion classifier on pseudo-labels + replay buffer. (3) Retrained classifier is validated on holdout subset of pseudo-labels. (4) If validation $F1 \geq 0.40$, classifier is replaced for subsequent events. Postcondition: Fusion classifier adapted to deployment distribution without label access.

3.5 Feasibility Study

Technical Feasibility

All required components are implementable using mature, production-stable Python libraries (scikit-learn, pandas, NumPy, LIME, FastAPI). The TON_IoT dataset is publicly available. The system requires no specialized hardware beyond a standard CPU-equipped workstation. Inference latency of $< 2\text{ms}$ per event (without LIME) is achievable on commodity hardware. Technical feasibility is confirmed.

Economic Feasibility

The system uses exclusively open-source libraries with no licensing costs. Dataset acquisition is free. Computational requirements are met by standard academic computing resources. The REST API implementation requires no cloud infrastructure for the current proof-of-concept deployment. Economic feasibility is confirmed.

Operational Feasibility

The system produces human-readable plain-English explanations for every detection, reducing the analyst expertise required to interpret outputs. The FSM state display provides continuous operational awareness. The REST API interface allows integration with existing SOC tooling. Operational feasibility is confirmed.

Chapter 4: Research Methodology

Following the research-track guidelines, this chapter replaces the conventional system design chapter with a description of the research methodology, experimental setup, and data analysis framework. CATF-IDS is a research project — its primary contributions are methodological rather than implementation-specific.

4.1 Dataset: TON_IoT

4.1.1 Dataset Overview

The TON_IoT dataset (Things-of-Network IoT) was published by Alsaedi et al. at the University of New South Wales (UNSW) in 2020. It is the only publicly available IIoT security benchmark providing co-temporal labeled data from four distinct data planes collected simultaneously in a single testbed environment. This co-temporality is essential for any multi-modal fusion evaluation: the four streams must reflect the same underlying system state at the same time, not data collected from different environments or time periods.

Modality	Source	Files	Features Used	Rows
Network Traffic	Zeek network flow logs	23 CSV	27 (engineered)	11,500
IoT Physical	7 device sensors (fridge, thermostat, GPS, etc.)	7 CSV	6 features	3,500
Linux System Logs	atop process and resource monitor	6 CSV	10 features	3,000
Windows Telemetry	Windows 7 and Windows 10 event logs	2 CSV	5 features	1,000
After Fusion	Temporally aligned via merge_asof	1 DataFrame	53 features	2,978

Table 4.1 — TON_IoT Dataset Composition

4.1.2 Attack Types and Distribution

The dataset includes nine labeled attack categories: network scanning, Denial of Service, brute-force authentication, data exfiltration, SQL/command injection, man-in-the-middle, ransomware, backdoor/C2 communication, and password attacks. After temporal fusion, the full deployment distribution contains 2,438 attack events (81.9%) and 540 normal events (18.1%). This extreme imbalance reflects the artificial laboratory conditions of the dataset generation, not real operational IIoT traffic, and is the primary source of the training-deployment distribution mismatch discussed throughout this report.

4.2 Data Preprocessing Pipeline

4.2.1 Balanced Loading Strategy

Each CSV file is loaded with a deliberate undersampling strategy: exactly 250 attack-labeled and 250 normal-labeled rows are selected per file. This balanced loading strategy forces the per-modality RF models to learn genuinely discriminative boundaries rather than class-imbalance shortcuts. The resulting merged training distribution is 50/50 across all modalities. While this requires undersampling attack examples, it is the theoretically correct approach for systems intended to operate in environments where attack prevalence may differ substantially from the training period.

4.2.2 Network Feature Engineering

Raw Zeek log fields are insufficient for attack discrimination. CATF-IDS computes 27 derived features organized into four layers:

- Statistical layer (4 features): net_duration, net_bytes, net_dbytes, net_pkts — raw traffic volume from Zeek flow fields.
- Signature layer (9 features): nine attack-specific scores computed by the NetworkAttackLibrary class, each measuring the degree to which the event matches a specific attack family signature (scan, DoS, brute-force, exfiltration, injection, MitM, ransomware, backdoor, password).
- Physics layer (12 features): behavioral physics features including payload asymmetry (exfiltration indicator), connection fan-out (scan indicator), traffic periodicity (beaconing indicator), burst ratio (DoS indicator), port entropy (scanning breadth), and inter-arrival time statistics.
- Risk fusion (1 feature): pre-classifier aggregate risk score from the NetworkRiskFusion component.

4.2.3 IoT Feature Engineering and Physical Delta

The critical derived feature from the IoT modality is phys_delta: the z-score deviation of each sensor reading from its own device-specific rolling baseline, computed over the past 50 readings.

This feature captures the degree to which a physical device's current reading deviates from its own historical norm — the primary signal of cyber-physical attack where an adversary manipulates the physical state of a device. The z-score normalization makes `phys_delta` comparable across different physical device types and measurement scales.

4.2.4 Temporal Fusion via `merge_asof`

Temporal alignment of four streams with different sampling frequencies requires a principled join strategy. CATF-IDS uses `pandas merge_asof` with `direction='backward'` — a nearest-neighbor backward join that, for each network event at timestamp t , finds the most recent record from each other stream at time $t' \leq t$. The backward direction is causally correct: it ensures that only information available at or before the decision moment is used, preventing data leakage from future observations. This is distinct from simple timestamp equality matching, which would discard the majority of cross-stream records.

4.3 Evaluation Methodology

4.3.1 The Dual FPR Reporting Protocol

A central methodological contribution of this work is the explicit dual FPR reporting protocol. CATF-IDS reports false positive rate under two distinct evaluation conditions:

- Scientific FPR (FPR = 0.111): measured on a balanced 50/50 held-out test set. This is the appropriate comparison condition against published literature, since most published systems either use balanced test sets or do not report FPR at all.
- Deployment FPR (FPR = 0.196): measured on the full 2,978-event stream with 82% attack prevalence. This is the honest operational metric that reflects what a SOC analyst would experience when running the system against the actual dataset distribution.

The gap between 0.111 and 0.196 is fully explained by the Energy FSM accumulation mechanism: 2,438 consecutive attack events push the system into ELEVATED/CRITICAL state, causing normal events arriving during this high-energy period to be escalated. The ablation study in Chapter 6 decomposes this effect precisely.

4.3.2 Stateful Five-Fold Cross-Validation

Standard k-fold cross-validation for adaptive systems is insufficient: if adaptive components (threshold, FSM energy, trust scores, memory cache) are not reset between folds, information from fold N contaminates the evaluation of fold N+1. CATF-IDS implements stateful cross-validation in which all adaptive components are fully reset at the start of each fold — new RF models, new fusion classifier, new threshold with fresh AutoTune, new FSM energy initialized to zero, new trust scores at default values, and empty memory cache.

Five folds are performed on the 1,080-row balanced dataset (864 training, 216 test per fold). Results are reported as mean $\pm$ standard deviation, with the coefficient of variation (CV%) measuring stability. The primary stability target is AUC standard deviation ≤ 0.02 .

4.3.3 Four-Level Aggregation Framework

After cross-validation, the five sets of fold parameters are aggregated into a single deployment configuration through a principled four-level strategy: (L1) AUC-weighted ensemble of five fold classifiers; (L2) F-beta weighted average threshold selection; (L3) stability-penalized trust initialization where modalities with high inter-fold trust variance receive conservative initialization; and (L4) FSM threshold calibration against the CV energy distribution. This framework ensures that the deployed system reflects the best statistical estimate of each parameter across the full validation run, not the output of any single fold.

4.4 Phase 2 Adaptation Design

The Phase 2 methodology implements a streaming adaptation protocol in which the identical 2,978-event stream is processed with labels completely hidden. Labels are revealed only at the end for evaluation — this is the "blindfold removal" protocol. Three mechanisms operate during the blindfolded run:

- **Confident Normal Tracker:** collects events scoring below $\tau_{\text{low}} \times 0.70$ while the FSM is in SAFE or SUSPICIOUS state. Designed to accumulate a clean sample of deployment normals for baseline recalibration.
- **FixMatch Consistency Gate:** for each candidate pseudo-label event, $K=3$ perturbed versions are generated with Gaussian noise ($\sigma=0.15$). The event is accepted as a pseudo-label only if all K perturbed versions predict the same class. This filters boundary events that would introduce label noise into retraining.
- **Fusion Classifier Self-Retraining:** when $\text{PL_MIN_RETRAIN}=100$ pseudo-labels have accumulated, the fusion classifier is retrained on the pseudo-labeled evidence vectors with a holdout validation check.

Chapter 5: Implementation

5.1 Implementation Environment and Setup

CATF-IDS is implemented as a single self-contained Python file: `ids_complete.py` (5,651 lines). This architectural decision is intentional — a single file eliminates dependency management complexity, simplifies deployment, and ensures complete reproducibility. All component classes, training loops, evaluation routines, and the Phase 2 adaptation mechanism are contained within this file.

The system is invoked from the command line with mode flags:

- `python ids_complete.py` — full Phase 1 run with five-fold CV and main pipeline evaluation.
- `python ids_complete.py --phase2` — run Phase 1 then execute Phase 2 blindfolded adaptation.
- `python ids_complete.py --fpr_ablation` — run the FPR ablation study (five variants).
- `python ids_complete.py --test` — self-test with synthetic data (no TON_IoT files required).

Hardware requirements are modest: a standard CPU workstation with 8GB+ RAM. No GPU is required. The full Phase 1 + CV pipeline completes in approximately 12–15 minutes on a modern CPU. Dataset CSV files are expected in configurable paths defined at the top of the file.

5.2 Nine-Layer Architecture Implementation

Layer 1 — Modal AI (class `LayerModels`)

Three independent `RandomForestClassifier` instances ($n_{\text{estimators}}=100$, $\text{class_weight}='balanced'$, $\text{random_state}=42$) are trained on modality-specific feature subsets. The `net_model` trains on `NET_FEATURES` (27 features) against `net_label` (the network-modality-specific label before consensus voting). Similarly for `iot_model` (`IOT_FEATURES`, 6 features) and `log_model` (`LOG_FEATURES`, 10 features). Each model uses its own `StandardScaler` fitted only on its modality's features. After training, `calibrate()` applies sigmoid correction ($\text{steepness}=5.0$) to all probability outputs to address Random Forest overconfidence.

Layer 2 — Behavioral Physics

The `NetworkBehaviorPhysics` class computes 12 behavioral physics features as derived statistics from raw Zeek log fields. The most diagnostic features are: $\text{phys_payload_asym} = (\text{src_bytes} - \text{dst_bytes}) / (\text{src_bytes} + \text{dst_bytes} + \epsilon)$, which captures the outbound dominance characteristic of data exfiltration; $\text{phys_fanout} = n_{\text{unique_destinations}} / (\text{duration} + \epsilon)$, which captures the

breadth of port/host scanning; and $\text{phys_periodicity} = 1 / (\sigma(\text{inter_arrival}) + \epsilon)$, which captures the regularity of command-and-control beaconing. The NetworkAttackLibrary computes nine additional signature scores, one per attack family, using hand-crafted threshold rules validated against known attack patterns in the literature.

Layer 3 — Fusion Meta-Classifier (class FusionClassifier)

A LogisticRegression model ($C=1.0$, $\text{max_iter}=1000$, $\text{class_weight}=\text{'balanced'}$) is trained on the 7-feature evidence vector $[\text{p_net}, \text{p_iot}, \text{p_log}, \text{L_net}, \text{L_iot}, \text{L_log}, \text{phys_delta}]$, where $\text{L_x} = \log(\text{p_x} / (1 - \text{p_x}))$ is the logit transformation of each calibrated modality probability. The evidence vector design provides the Logistic Regression with both the natural probability space and the unbounded log-odds space for each modality signal, enabling the model to learn different non-linear responses to the same signal in different regimes. The learned weights confirm that network probability ($\text{p_net} \approx +1.92$) and network log-odds ($\text{L_net} \approx +1.63$) are the dominant positive contributors, followed by IoT probability ($\text{p_iot} \approx +1.27$).

Layer 4 — Modality Trust Tracker (class ModalityTrustTracker)

Trust is tracked per modality as a scalar in $[\text{min_trust}, 1.0]$. After each event decision, each modality's prediction ($\text{p} > 0.5 \rightarrow \text{attack}$, else normal) is compared to the final decision. Agreement increases trust by $+0.015$; disagreement decreases it by -0.008 . The asymmetric update (award $<$ penalty in absolute terms) ensures that trust is conservative: recovery from a poor period requires roughly twice as many correct predictions as incorrect ones. Trust is applied by dampening each modality probability toward 0.5: $\text{p_trusted} = \tau \times \text{p_raw}$, where $\tau \in [\text{min_trust}, 1.0]$. Phase 1 final trust values: $\text{net}=1.000$, $\text{iot}=0.986$, $\text{log}=1.000$, $\text{win}=0.700$ (fixed ceiling due to limited feature count).

Layer 5 — Policy Engine (class PolicyEngine)

The policy engine implements three-level symbolic rule checking: (L1) per-field threshold violations with severity proportional to degree of violation; (L2) amplification when multiple fields in the same domain simultaneously violate thresholds; (L3) cross-domain correlation rules encoding known multi-plane attack signatures (e.g., high net_bytes + high disk_write = potential exfiltration). Five forced policies bypass ML entirely: $\text{page_faults_maj} > 300$ (exploit/injection), $\text{mem_vgrow} > 3000 + \text{CPU} > 85\%$ (malware), $\text{disk} + \text{CPU} + \text{net}$ (ransomware signature), $\text{abs}(\text{phys_delta}) > 15.0$ (sensor tampering), and $\text{src_bytes} > 500,000$ (exfiltration). In Phase 1, 70 events (2.4%) triggered forced policies.

Layers 6 and 7 — Adaptive and Contextual Threshold

AdaptiveThreshold maintains tau_high and tau_low updated by the EWMA: $\mu_t = 0.05 \times s_t + 0.95 \times \mu_{\{t-1\}}$. When $|\mu_t - \text{baseline_mean}| > 0.08$ (the drift sensitivity threshold), tau_high is adjusted by $\text{clamp}(0.5 \times \text{drift}, -0.04, +0.04)$ from its base value. This clamp bounds the per-step adjustment, providing adversarial robustness against score-stream manipulation.

ContextualThresholdEngine computes per-event adjustments: $T_{\text{adaptive}} = T_{\text{base}} - \alpha \cdot D + \delta \cdot U + \text{trust_term}$, where D is the current drift magnitude, U is the standard deviation of the three modality probabilities (layer disagreement), and $\text{trust_term} = (1 - \text{mean_trust}) \times 0.04$. The adjustment is passed through $\tanh(x/0.12) \times 0.12$ before application, bounding the maximum per-event shift to ± 0.12 .

Layer 8 — Energy FSM (class ThreatStateMachine)

The FSM maintains a running energy score updated as $E_t = 0.88 \times E_{\{t-1\}} + 0.12 \times R_t$, where $R_t = (\text{event_level}/4) + 0.3 \times \text{cluster_risk} + 0.2 \times \text{campaign_risk}$. State transitions: SAFE ($E < 0.28$), SUSPICIOUS ($0.28 \leq E < 0.42$), ELEVATED ($0.42 \leq E < 0.62$), CRITICAL ($E \geq 0.62$). The confidence gate implements: if $s_{\text{level}} \geq 2$ AND $\text{fusion_score} > \text{tau_low}$: $\text{dec} = 1$. This gate was the key innovation reducing deployment FPR from 0.478 to 0.196 while preserving all six

campaign detections. The gate ensures the FSM can only escalate events that the ML model itself considers suspicious (above τ_{low}), preventing the FSM from overriding confident-normal events during high-energy periods.

Layer 9 — Behavioral Memory (class ExperienceMemory + BehavioralThreatMemory)

The experience memory implements three tiers: (1) Signature cache: events are fingerprinted by $(\text{round}(p_{net},2), \text{round}(p_{iot},2), \text{round}(p_{log},2), \text{round}(phys_delta,1))$; the first occurrence runs the full ML pipeline and stores the decision; subsequent identical fingerprints are served directly from cache. Phase 1 cache: 588 entries, 85.1% hit rate. (2) DBSCAN clustering: after the warmup period, DBSCAN($\epsilon=0.5$, $\text{min_samples}=5$) clusters the accumulated evidence vectors. Cluster risk scores (fraction of cluster events classified as attack, scaled by cluster size and capped at 75%) are used as context signals for novel events. Phase 1: 24 clusters. (3)

BehavioralThreatMemory maintains behavioral intent profiles per event and matches sequences against five known attack progression templates (recon_to_exploit , recon_to_exfil , beacon_to_exfil , dos_campaign , lateral_spread). Phase 1: 6 campaigns detected.

5.3 Phase 2 Implementation

Phase 2 is invoked by the `--phase2` flag and runs immediately after Phase 1 completes. The same 2,978-event stream is processed again with labels completely withheld from all decision-making components. The label array is kept separate and only used at the very end of the run for metric computation — the "blindfold removal" at the conclusion of the experiment.

The adaptive pseudo-label thresholds are computed per-event rather than using fixed values:

$\text{PL_ATK_THRESH} = \text{adaptive_tau.tau_high} + 0.24$ and $\text{PL_NRM_THRESH} =$

$\text{adaptive_tau.tau_low} \times 0.70$. This design ensures that the pseudo-label confidence margin remains constant relative to the current detection boundary, which moves throughout the inference run as EWMA drift evolves. The FixMatch consistency gate then applies $K=3$ Gaussian perturbations ($\sigma=0.15$) to the evidence features of each candidate, rejecting boundary events where any perturbation flips the predicted class. In Phase 2, 256 candidates were tested with 241 (94%) accepted and 15 (5.9%) rejected as boundary events.

The ExplainabilityEngine class provides three-level per-event explanations: (1) event level — decision reason parsed from compound reason strings (e.g., "[ELEVATED] FUSION_ALERT: Attack..."), modality agreement summary, fusion score position relative to adaptive thresholds; (2) risk breakdown — contribution of each signal component (N, I, L, W, C, T, P) to unified risk R with bar chart visualization; (3) behavioral intent narrative — dominant attack type from the intent profile with confidence percentage and secondary indicators.

Chapter 6: Testing and Evaluation

6.1 Testing Strategy

CATF-IDS is evaluated through four complementary testing approaches, each answering a distinct scientific question:

- Unit testing (`--test` flag): synthetic data generation (`save_synthetic_csvs`) produces 2,600 events with five attack types and one normal type. The full pipeline runs on this synthetic data, verifying all components function without errors in the absence of the TON_IoT dataset.
- Cross-validation: five-fold stateful CV on 1,080 balanced rows provides stability-validated performance estimates with confidence intervals. This tests whether the results are robust to the choice of training/test split.
- Full deployment evaluation: the main pipeline runs on all 2,978 fused events under the realistic 82% attack distribution. This tests operational performance.

- Ablation study (--fpr_ablation): five variants of the pipeline with individual adaptive components disabled. This tests the causal contribution of each architectural component to the observed FPR.

6.2 Cross-Validation Results

Table 6.1 reports the five-fold cross-validation results. Each fold resets all adaptive components completely. The reported confidence intervals use the t-distribution with 4 degrees of freedom ($T_CRIT_95 = 2.776$ at 95% confidence).

Metric	Mean	Std Dev	CV%	95% CI
ROC-AUC	0.9218	0.0134	1.5%	[0.884, 0.960]
F1-Score	0.7323	0.0535	7.3%	[0.584, 0.881]
Precision	0.8053	0.0567	7.0%	[0.648, 0.963]
Recall	0.6759	0.0736	10.9%	[0.472, 0.879]
FPR	0.1667	0.0580	34.8%	[0.006, 0.328]
tau_high (mean)	0.360	0.020	5.6%	[0.305, 0.415]

Table 6.1 — Five-Fold Cross-Validation Results (1,080 balanced rows, stateful reset per fold)

The ROC-AUC coefficient of variation of 1.5% — the lowest of any reported metric — confirms that the model's discriminative ability is stable across all five independent fold assignments. The higher FPR coefficient of variation (34.8%) is expected: with only 216 events per test fold and approximately 108 normal events, each FPR measurement has high statistical variance. The 95% confidence interval [0.006, 0.328] honestly reflects this uncertainty.

6.3 Main Run — Phase 1 Full Deployment Evaluation

The main pipeline trains on 756 balanced rows (70% of the balanced_fused DataFrame) and evaluates on the full 2,978-event stream:

Metric	Phase 1 Value	Note
F1-Score	0.8268	Full deployment (82% attack) test set
Precision	0.9442	Very few false alarms relative to detections
Recall	0.7354	Detects 73.5% of all 2,438 attack events
ROC-AUC	0.9318	Threshold-independent discrimination ability
FPR (deployment)	0.1963	Operational metric — 82% attack stream
FPR (scientific)	0.111	Balanced 50/50 test — comparable to literature
Accuracy	0.75	Lower than literature due to honest evaluation
Campaigns detected	6	Recon → Access → Persistence × 6
Memory cache entries	588	85.1% of decisions served from cache
Contextual adjustments	309	Per-event threshold tuning during inference
Online learner retrains	1	At 527 pseudo-labels; holdout F1 = 1.0

Table 6.2 — Phase 1 Full Deployment Evaluation Results (2,978 events, 82% attack)

6.4 FPR Ablation Study

The ablation study addresses the key question raised by the deployment FPR of 0.196: what does each adaptive component contribute to the FPR, and what is the cost of removing it? Table 6.3 reports the five ablation variants:

Variant	FPR	F1	Recall	ΔFPR	Capability Lost
Full CATF-IDS	0.494	0.824	0.778	baseline	Nothing
No drift (alpha=0)	0.494	0.824	0.778	+0.000	Environmental adaptation
No FSM override	0.133	0.845	0.754	−0.361	Campaign detection
Static threshold	0.483	0.825	0.777	−0.011	All threshold adaptation
Like published papers	0.122	0.846	0.754	−0.372	Everything novel

Table 6.3 — FPR Ablation Study (Five Variants of the Full Pipeline)

The ablation study yields three key findings: (1) The FSM override is entirely responsible for 73% of the deployment FPR — removing it reduces FPR by 0.361. (2) Removing adaptive threshold drift contributes zero FPR reduction — drift is not the source of the FPR. (3) The cost of removing all adaptive components (the "like published papers" variant) is 59 additional missed attacks (recall: 0.778 $\rightarrow$ 0.754, a 3.1% recall reduction) and complete loss of multi-stage campaign detection capability. Published systems reporting FPR $\approx$ 0.12 on TON_IoT are implicitly operating in this last variant — without temporal escalation — and their low FPR comes at the undisclosed cost of missing campaigns.

6.5 Phase 2 Blindfolded Adaptation Results

Phase 2 runs the identical system on the same 2,978-event stream with all labels hidden. Table 6.4 compares Phase 1 and Phase 2 on the same evaluation set:

Metric	Phase 1	Phase 2 (Blindfolded)	Change
F1-Score	0.8268	0.8496	+ 0.0228 ✓
Precision	0.9442	0.9559	+ 0.0117 ✓
Recall	0.7354	0.7646	+ 0.0292 ✓
ROC-AUC	0.9318	0.9361	+ 0.0043 ✓
FPR	0.1963	0.1593	− 0.0370 ✓
Pseudo-labels collected	—	241 (94% pass rate)	15 rejected (5.9%)
Consistency gate tested	—	256 candidates	5.9% boundary rejection rate
Fusion retrains	1	2	More deployment data available

Table 6.4 — Phase 1 vs Phase 2 Blindfolded Adaptation (2,978 events)

Every metric improved simultaneously — an unusual result in adaptive IDS systems where FPR improvements typically trade against recall. The improvement is driven by pseudo-label self-retraining: 241 high-confidence pseudo-labels (after consistency filtering) from the deployment stream allowed the fusion classifier to adapt its weights from the balanced training distribution to the actual deployment distribution. The confidence gate rejection rate of 5.9% confirms that the FixMatch consistency mechanism is functioning as designed — filtering genuinely boundary-ambiguous candidates.

The confident normal tracker did not trigger baseline recalibration: only 9 events scored below the confident-normal threshold during Phase 2. This is a dataset artifact — the 82% attack density keeps the FSM in ELEVATED/CRITICAL state throughout most of the stream, preventing the tracker from accumulating the 150 required samples. In a realistic IIoT deployment with 5–10% attack density, the tracker would accumulate confident normals rapidly, enabling baseline recalibration and further FPR reduction.

6.6 Limitations and Known Issues

- Dataset attack density (82%) prevents the Phase 2 confident normal tracker from reaching its minimum sample threshold. Baseline recalibration — the most effective adaptation mechanism for deployment FPR reduction — requires a more operationally realistic attack prevalence.
- The deployment FPR of 0.196 is significantly higher than the scientific FPR of 0.111. This gap is fully explained and documented (FSM accumulation under sustained attack streams), but it may limit acceptance in environments with very strict false alarm rate requirements.
- Cross-dataset validation is not possible in the four-modality configuration because no alternative dataset provides co-temporal data from all four required data planes.

- The LIME explanation generation adds approximately 50–200ms per explained event. For high-throughput environments, LIME should be applied selectively to high-severity detections only.

Chapter 7: Conclusion and Future Work

7.1 Summary of Achievements

CATF-IDS achieves all six project objectives stated in Chapter 1. The system simultaneously fuses network traffic, IoT physical sensor readings, Linux system logs, and Windows telemetry — the first published IDS to do so on the TON_IoT benchmark. The nine-layer governance architecture implements the separation principle between detection and escalation, preventing the alert-fatigue feedback loop. Adaptive threshold governance with EWMA drift tracking, contextual per-event adjustment, and the confidence gate mechanism collectively reduce deployment FPR by 58.9% compared to the system without the gate. The Energy FSM detects six multi-stage attack campaigns across the 2,978-event evaluation stream. Five-fold stateful cross-validation with four-level parameter aggregation validates the result ($AUC = 0.922 \pm 0.013$) against the alternative hypothesis of lucky data splitting. Phase 2 blindfolded adaptation improves all five evaluation metrics simultaneously without any label access.

The ablation study constitutes a particularly significant contribution: it rigorously quantifies the trade-off between adaptive temporal intelligence and false positive rate, demonstrating that the deployment FPR gap between CATF-IDS and published single-modality systems is the direct, documented, and quantified cost of the multi-stage campaign detection capability that distinguishes CATF-IDS from all prior work.

7.2 Lessons Learned

- The separation principle is architecturally critical. Early prototypes that coupled the detection threshold to the FSM energy created runaway escalation feedback loops. Strictly separating detection (threshold-based, responding to statistical signals) from escalation (FSM-based, responding to contextual signals) resolved this and enabled clean ablation decomposition.
- Honest FPR reporting requires dual measurement. Reporting only the scientific FPR on a balanced test set would have hidden the operational reality of the system. Reporting only the deployment FPR would have made the system appear weak compared to published systems that do not report FPR at all. Both are necessary and neither is sufficient alone.
- The ablation study is more valuable than any additional architectural complexity. Understanding exactly why FPR is elevated — and documenting the cost of lowering it — is a stronger contribution than attempting to engineer away the FPR at the cost of hiding the explanation.
- Pseudo-label self-training (Phase 2) works even in the adversarial 82% attack density environment. Despite the confident normal tracker failing to trigger baseline recalibration, the consistency-gated pseudo-label mechanism delivered consistent all-metric improvement. The FixMatch consistency gate (5.9% boundary rejection rate) confirmed that the mechanism correctly identifies and filters noisier pseudo-labels.
- Temporal fusion alignment (merge_asof backward) is not obvious but critical. An initial implementation using timestamp equality matching discarded 97% of cross-stream events. The backward nearest-neighbor join approach recovers all events while preserving causal ordering.

7.3 Future Enhancements

Short-term (0–6 months)

- Cross-dataset validation on a future multi-modal IIoT benchmark. Several research groups are developing datasets with two or three modalities; any dataset providing network + physical data simultaneously would enable partial validation.
- Real-time streaming deployment with Kafka or MQTT integration, replacing the current batch-file processing with live event streams from actual IIoT infrastructure.
- Formal concept drift detection using ADWIN or Page-Hinkley tests to trigger adaptation explicitly when the deployment distribution has shifted significantly, rather than relying on EWMA drift tracking alone.

Medium-term (6–18 months)

- Per-modality autoencoder anomaly detection trained on confident deployment normals, providing an additional unsupervised anomaly signal to complement the supervised fusion score.
- GNN-based representation learning on the temporal event graph, enabling structural pattern recognition across longer time windows than the Energy FSM's decay constant supports.
- Federated learning adaptation for multi-site IIoT deployments where sharing raw data between sites is prohibited, but model updates can be aggregated.

Long-term (18+ months)

- Full cyber-physical simulation integration with industrial control system testbeds (Mininet, GNS3 + hardware-in-the-loop), enabling evaluation against live attack scenarios rather than recorded datasets.
- Formal verification of the separation principle using model-checking techniques to prove the absence of detection-escalation feedback loops under all input conditions.

References

- [1] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in Proc. Military Communications and Information Systems Conference (MilCIS), Canberra, Australia, 2015, pp. 1–6.
- [2] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, and A. Anwar, "TON IoT telemetry dataset: A new generation dataset of IoT and IIoT for data-driven intrusion detection systems," IEEE Access, vol. 8, pp. 165 130–165 150, Sep. 2020.
- [3] X. Shou, J. Li, Y. Chen, Y. Liu, and L. Ye, "ROAST-IoT: A novel range-oriented attention convolutional neural network for scalable attacks detection in IoT networks," in Proc. ACSAC, 2023.
- [4] N. Moustafa and L. Turnbull, "Explainable intrusion detection for cyber physical systems via transformer-based stacked autoencoder," IEEE Trans. Ind. Informatics, vol. 19, no. 2, pp. 1024–1034, Feb. 2023.
- [5] K. Sohn, D. Berthelot, N. Carlini, Z. Zhang, H. Zhang, C. A. Raffel, E. D. Cubuk, A. Kurakin, and C.-L. Li, "FixMatch: Simplifying semi-supervised learning with consistency and confidence," in Proc. NeurIPS, 2020.
- [6] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you?: Explaining the predictions of any classifier," in Proc. ACM SIGKDD, San Francisco, CA, 2016, pp. 1135–1144.
- [7] D. Yarowsky, "Unsupervised word sense disambiguation rivaling supervised methods," in Proc. ACL, Cambridge, MA, 1995, pp. 189–196.
- [8] L. Breiman, "Random forests," Mach. Learn., vol. 45, no. 1, pp. 5–32, Oct. 2001.
- [9] D. Ester, H. P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in Proc. KDD, Portland, OR, 1996, pp. 226–231.

- [10] D. J. Bernstein, P. Birkner, M. Joye, T. Lange, and C. Peters, "Twisted Edwards curves," in Proc. AFRICACRYPT, 2008. [Referenced for energy model notation conventions]
- [11] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, "Survey of intrusion detection systems: Techniques, datasets and challenges," *Cybersecurity*, vol. 2, no. 1, pp. 1–22, 2019.
- [12] M. A. Ferrag, L. Shu, X. Yang, A. Derhab, and L. Maglaras, "DeepCoin: A novel deep learning and blockchain-based energy exchange framework for smart grids," *IEEE Trans. Eng. Manag.*, vol. 67, no. 4, pp. 1285–1297, Nov. 2020.
- [13] Y. Xie, Z. Liu, C. Gao, D. Wang, G. Chen, and J. Shu, "CROP: A method of adaptive online detection for concept drifts in network traffic classification," *IEEE Trans. Netw. Serv. Manag.*, vol. 18, no. 2, pp. 1731–1748, Jun. 2021.
- [14] G. Shu, Y. Peng, J. Ma, and M. Liu, "SSTD: Self-supervised traffic differentiation for IoT intrusion detection using graph neural networks," *IEEE Internet Things J.*, vol. 11, no. 4, pp. 5874–5886, Feb. 2024.
- [15] X. Yao, X. Chen, Y. Hu, J. Abawajy, and M. Hassan, "A semi-supervised learning approach to IoT intrusion detection with limited labeled data," *IEEE Trans. Netw. Serv. Manag.*, vol. 21, no. 1, pp. 1234–1248, Feb. 2024.
- [16] F. Hussain, R. Hussain, S. A. Hassan, and E. Hossain, "Machine learning in IoT security: Current solutions and future challenges," *IEEE Commun. Surv. Tut.*, vol. 22, no. 3, pp. 1686–1721, 3rd Quart. 2020.
- [17] T. T. Nguyen and V. J. Reddi, "Deep reinforcement learning for cyber security," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 8, pp. 3779–3795, Aug. 2023.
- [18] G. Apruzzese, M. Colajanni, L. Ferretti, A. Guido, and M. Marchetti, "On the effectiveness of machine and deep learning for cyber security," in Proc. CRITIS, 2018, pp. 371–378.
- [19] National Institute of Standards and Technology, "Guide to industrial control systems (ICS) security," NIST Special Publication 800-82 Rev. 2, U.S. Dept. Commerce, 2015.
- [20] European Union Agency for Cybersecurity (ENISA), "Cybersecurity for the IoT: Challenges and recommendations," ENISA Report, Nov. 2021.

Appendix A: System Execution Guide

A.1 Prerequisites

- Python 3.8 or higher
- Required packages: `pip install scikit-learn pandas numpy lime fastapi uvicorn matplotlib`
- TON_IoT dataset CSV files organized in the directory structure defined in `ids_complete.py` (lines 4–30)

A.2 Running the System

```
# Phase 1 full evaluation
python ids_complete.py
# Phase 2 blindfolded adaptation
python ids_complete.py --phase2
# FPR ablation study (5 variants)
python ids_complete.py --fpr_ablation
# Self-test without dataset files
python ids_complete.py --test
```

A.3 REST API

Start the API server: `python ids_complete.py --serve`. Available endpoints: POST `/predict` (submit event for classification), GET `/status` (system state, trust scores, FSM energy, cache size), GET `/campaigns` (detected campaign list with narratives), GET `/health` (system health check).

Appendix B: Phase 2 Consistency Regularization Experiment Results

Table B.1 summarizes the three Phase 2 configurations tested during development:

Configuration	FPR	F1	Prec	Recall	AUC
Phase 1 baseline	0.1963	0.8268	0.9442	0.7354	0.9318
Phase 2 — pseudo-labels only (static gate)	0.1593	0.8496	0.9559	0.7646	0.9365
Phase 2 — FixMatch consistency (selected)	0.1593	0.8496	0.9559	0.7646	0.9361

Table B.1 — Phase 2 Configuration Comparison

The FixMatch consistency configuration ($\tau_{\text{high}} + 0.24$ primary threshold, $\sigma=0.15$ noise, $K=3$ perturbations) achieves identical operational metrics to the static gate while implementing the theoretically principled FixMatch-style boundary filtering. The consistency check rejected 15 of 256 candidates (5.9%), confirming the gate is functioning as designed. The slight AUC difference (0.9361 vs 0.9365) is within statistical noise at this scale.

Appendix C: Extended User Manual & Configuration Reference

This appendix expands on the quick-start guide in Appendix A with a full command reference, REST API reference, and the internal configuration values that govern CATF-IDS behavior at deployment time.

C.1 Installation

CATF-IDS is a single-file Python 3.8+ application. Install dependencies with:

```
pip install pandas numpy scikit-learn joblib matplotlib
pip install seaborn fastapi uvicorn lime
```

The lime package is optional; if it is not installed, LIME-based explainability is skipped automatically and the rest of the pipeline runs unaffected.

C.2 Command-Line Reference

Flag	Effect
(none)	Run the full Phase 1 pipeline on the configured TON_IoT dataset paths.
--test	Run a self-test on synthetic data that mirrors the TON_IoT schema; no CSV files required.
--phase2	After Phase 1 completes, run Phase 2 blindfolded deployment adaptation on the same event stream.
--fpr_ablation	Run the five-variant FPR ablation study comparing the full adaptive system against static-threshold baselines.
--quick_compare	Run a targeted 6-combination aggregation-strategy comparison instead of the full CV grid search.
--serve	Start the FastAPI inference server (requires models already trained and saved to models/).
--n <int>	Rows sampled per modality file (default 10,000); lower this for faster iteration on limited hardware.

C.3 REST API Endpoint Reference

Once started with --serve, the API listens on <http://127.0.0.1:8000> with interactive documentation at /docs. Key endpoints:

Endpoint	Method	Purpose
/health	GET	Reports whether models are loaded, cache size, active context, and incident counter.
/analyze	POST	Submit a single event (network, IoT, disk, CPU features) for real-time classification.
/intent/apply	POST	Apply a natural-language operational policy (e.g. "scheduled backup") that rescales incoming signals before scoring.

/intent/context	GET	View the currently active intent context and recent policy history.
/memory/stats	GET	Cache size, evidence log length, pseudo-label count, and DBSCAN cluster count.
/memory/clusters	GET	List current DBSCAN cluster IDs, sizes, and risk levels.
/memory/run_dbscan	POST	Force a re-clustering pass over the accumulated evidence log.
/learn/retrain	POST	Trigger an online-learning retrain cycle (incremental or batch mode).
/learn/status	GET	Current online-learner state: retrain count, pseudo-labels used, last result.
/learn/history	GET	Full history of past retrain events.

C.4 Key Configuration Values

These constants govern the adaptive components described in Chapters 4 and 5. They can be tuned without retraining the underlying Random Forest or fusion models.

Component	Parameter	Default	Effect
AdaptiveThreshold	beta	1.5	F-beta weighting; beta>1 favors recall over precision when learning tau.
AdaptiveThreshold	MAX_SHIFT_PER_STEP	0.04	Caps how far tau_high can move in a single EWMA update (adversarial hardening).
AdaptiveThreshold	MIN / MAX_TAU_HIGH	0.35 / 0.90	Hard floor and ceiling — threshold can never collapse or saturate beyond these.
ContextualThresholdEngine	ALPHA / DELTA	0.40 / 0.20	Weight of drift (D) vs layer disagreement (U) in the live threshold equation.
ThreatStateMachine	ENERGY_DECAY	0.88	Per-event decay of accumulated escalation energy.
ThreatStateMachine	THRESHOLDS	CRITICAL>=0.62, ELEVATED>=0.42, SUSPICIOUS>=0.28	Energy levels mapped to the 5-level threat verdict.

ModalityTrustTracker	min_trust	0.45	Floor trust value — no modality can ever be fully ignored, even if persistently noisy.
ConsistencyChecker (Phase 2)	n_perturb / noise_std	3 / 0.15	FixMatch-style perturbation count and Gaussian noise scale for pseudo-label filtering.

C.5 Typical Operational Workflow

- Train Phase 1 models on historical labeled data: `python ids_complete.py`
- Validate adaptive components with the FPR ablation study: `python ids_complete.py --fpr_ablation`
- Deploy in blindfolded mode to confirm the system self-calibrates without labels: `python ids_complete.py --phase2`
- Start the REST API for live integration with a SOC dashboard: `python ids_complete.py --serve`
- Apply operational context during known maintenance windows via POST `/intent/apply` to reduce false positives without lowering the detection threshold globally.

Appendix D: Source Code Excerpts

The complete system is implemented as a single file, `ids_complete.py` (5,651 lines). The excerpts below are drawn directly from that file and illustrate the core adaptive-governance components discussed in Chapters 4 and 5. Non-essential logging and error-handling lines have been omitted for brevity where marked "...".

D.1 Adaptive Threshold — Learned Boundary with EWMA Drift Correction

```
class AdaptiveThreshold:
    MAX_SHIFT_PER_STEP = 0.04 # max tau change per EWMA update
    MIN_TAU_HIGH = 0.35 # hard floor
    MAX_TAU_HIGH = 0.90 # hard ceiling

    def fit(self, y_true, y_scores):
        best_tau, best_fb = 0.5, 0.0
        for tau in np.arange(0.30, 0.85, 0.01):
            preds = (y_scores >= tau).astype(int)
            fb = fbeta_score(y_true, preds, beta=self.beta, zero_division=0)
            if fb > best_fb:
                best_fb = fb; best_tau = tau
        self.tau_high = round(float(best_tau), 3)
        # tau_low always stays >= 0.15 below tau_high (uncertainty zone)
        raw_low = round(max(0.05, 1.0 - float(best_tau) - 0.10), 3)
        self.tau_low = min(raw_low, self.tau_high - 0.15)
        ...

    def update(self, fusion_score):
        self._ewma_score = (self.ewma_alpha * fusion_score
                             + (1 - self.ewma_alpha) * self._ewma_score)
        drift = self._ewma_score - self._baseline_mean
        if abs(drift) > self.drift_sensitivity:
            clamped_shift = float(np.clip(drift * 0.5,
                                           -self.MAX_SHIFT_PER_STEP,
                                           self.MAX_SHIFT_PER_STEP))
            new_h = float(np.clip(self._base_tau_high + clamped_shift,
                                   self.MIN_TAU_HIGH, self.MAX_TAU_HIGH))
            self.tau_high = round(new_h, 3)
```

D.2 Unified Risk Scorer — Weighted Multi-Signal Fusion

```
class UnifiedRiskScorer:
    WEIGHTS = {
        "network": 0.35, "iot": 0.20, "linux": 0.15,
        "windows": 0.10, "cluster": 0.10,
        "temporal": 0.05, "policy": 0.05,
    }

    def score(self, p_net, p_iot, p_log, phys_delta,
              system_stress, cluster_risk, row_features, adaptive_tau=None):
        w = self.WEIGHTS
        N, I, L, C = float(p_net), float(p_iot), float(p_log), float(cluster_risk)
        W = min(float(system_stress) / 100.0, 1.0) if system_stress > 0 else float(p_log)
        P_risk, flags = self.policy.evaluate(row_features)

        R = (w["network"]*N + w["iot"]*I + w["linux"]*L +
              w["windows"]*W + w["cluster"]*C +
              w["temporal"]*T + w["policy"]*P_risk)
```



```

# Risk floor: policy violations set a minimum risk
if P_risk > 0.25: R = max(R, 0.45) # suspicious zone minimum
if P_risk > 0.45: R = max(R, 0.65) # attack zone if critical
return float(np.clip(R, 0.0, 1.0)), breakdown, flags

```

D.3 Hierarchical Decision Engine

```

def hierarchical_decision(p_net, p_iot, p_log, phys_delta,
                        fusion_score, evidence_vec, memory, adaptive_tau=None):
    # L4: Policy hard overrides (bypass ML entirely)
    if p_net > 0.95:
        return 1, "OVERRIDE: Network critical (p_net>0.95)", float(p_net)
    if p_iot > 0.90 and abs(phys_delta) > 5.0:
        return 1, f"OVERRIDE: Physical tampering (delta={phys_delta:.1f})", float(p_iot)

    # L5: Adaptive global threshold (learned tau + EWMA drift)
    tau_h = adaptive_tau.tau_high if adaptive_tau else TAU_HIGH
    tau_l = adaptive_tau.tau_low if adaptive_tau else TAU_LOW
    if fusion_score > tau_h:
        return 1, f"FUSION_ALERT: Attack (score={fusion_score:.2f})", float(fusion_score)
    if fusion_score < tau_l:
        return 0, f"FUSION_NORMAL: Baseline (score={fusion_score:.2f})", float(1-fusion_score)

    # Suspicious zone: cluster + outlier resolution
    cid, cluster_risk, dist = memory.find_closest_cluster(evidence_vec)
    if cid != -1 and cluster_risk > 0.70:
        return 1, f"CLUSTER_MATCH: Attack cluster {cid}", cluster_risk
    if is_outlier(evidence_vec, memory):
        return 1, "ANOMALY: Novel pattern outside correlation space", float(fusion_score)
    return 0, "SUSPICIOUS_SAFE: Uncertain, close to safe cluster", float(1-fusion_score)

```

D.4 Network Attack Signature Scoring

```

ATTACK_SIGNATURES = {
    "scan": {
        "description": "Port scan / host sweep",
        "rules": [("duration", "lt", 0.5), ("src_bytes", "lt", 500),
                  ("dst_bytes", "lt", 200), ("src_pkts", "lt", 5)],
        "conn_state_hit": ["S0", "REJ", "RST0"],
        "weight": 1.0,
    },
    "exfiltration": {
        "description": "Large outbound, sustained connection",
        "rules": [("src_bytes", "gt", 100_000), ("dst_bytes", "lt", 10_000),
                  ("duration", "gt", 5.0)],
        "weight": 1.0,
    },
    ... # 7 more families: dos, brute_force, injection, mitm,
        # ransomware, backdoor, password
}

```

```

class NetworkAttackLibrary:
    def _score_signature(self, row, sig):
        matched = 0
        for col, op, threshold in sig.get("rules", []):
            val = float(row.get(col, 0) or 0)
            if op == "gt" and val > threshold: matched += 1
            elif op == "lt" and val < threshold: matched += 1
        if str(row.get("conn_state", "")).upper() in sig.get("conn_state_hit", []):
            matched += 0.5
        max_possible = len(sig["rules"]) + 1.5
        return float(np.clip((matched/max_possible) * sig["weight"], 0.0, 1.0))

```

D.5 Threat State Machine — Energy-Based Escalation

```
class ThreatStateMachine:
    ENERGY_DECAY = 0.88
    FORCED_STICKY = 3
    THRESHOLDS = {3: 0.62, 2: 0.42, 1: 0.28}

    def update(self, event_level, cluster_risk=0.0, policy_risk=0.0):
        if event_level == 4:
            self.state = 4; self._forced_ttl = self.FORCED_STICKY
            self.energy = 1.0
            return self.state
        level_score = event_level / 4.0
        escalation = 0.3*cluster_risk + 0.2*policy_risk
        current_risk = float(np.clip(level_score + escalation, 0.0, 1.0))
        self.energy = float(np.clip(
            self.ENERGY_DECAY*self.energy + (1-self.ENERGY_DECAY)*current_risk, 0, 1))
        new_state = 0
        for lvl in sorted(self.THRESHOLDS, reverse=True):
            if self.energy >= self.THRESHOLDS[lvl]:
                new_state = lvl; break
        self.state = new_state
        return self.state
```

Appendix E: Representative Raw Data Samples

The tables below show representative example rows illustrating the structure of each modality's raw input before preprocessing. Values are illustrative and constructed to match the true TON_IoT column schema used by the pipeline (Sections 4.2-4.3); they are not a direct export of a specific experimental run.

E.1 Network Modality (Network_dataset_X.csv)

ts	src_ip	dst_ip	dst_port	proto	service	duration	src_bytes	dst_bytes	conn_state	label	type
1558015140	192.168.1.10	192.168.1.20	80	tcp	http	0.23	534	1280	SF	0	normal
1558015141	10.0.0.8	192.168.1.20	22	tcp	ssh	0.01	65	0	S0	1	scanning
1558015142	172.16.0.5	192.168.1.30	445	tcp	smb	1.84	4812	902	REJ	1	ransomware

Fields used directly by the pipeline: src_bytes, dst_bytes, duration, proto, service, and conn_state (Section 5.2, preprocess_network). src_ip is used only for per-session behavioral physics (fan-out, lateral movement); it is not fed to the Random Forest directly.

E.2 IoT Modality (device-specific CSVs)

ts	fridge_temperature	temp_condition	label
1558015200	3.9	Normal	normal
1558015201	4.1	Normal	normal
1558015202	18.7	High	backdoor

IoT_Fridge.csv — representative rows

ts	latitude	longitude	speed	battery	label
1558015200	-37.813	144.963	48	92	normal
1558015201	-37.814	144.962	0	91	normal
1558015202	0	0	120	14	attack

IoT_GPS_Tracker.csv — representative rows

Each of the seven IoT device types (Fridge, Garage Door, GPS Tracker, Modbus, Motion Light, Thermostat, Weather) has its own native columns; the device-aware normalizer (Section 5.2, normalize_iot_device) maps each device's primary signal onto the universal current_temp / temp_con_enc representation before fusion.

E.3 Linux Log Modality (linux_disk_*.csv, linux_memory*.csv, Linux_process_*.csv)

ts	PID	WRDSK	MEM	CPU	MINFLT	CMD	label
1558015300	342	12	0.6	4.3	42	bash	normal

1558015301	520	1024	4.5	91.2	805	encrypt	attack
1558015302	111	8	0.5	2.4	15	systemd	normal

The extreme WRDSK / CPU / MINFLT combination in row 2 is exactly the pattern the ransomware forced-policy rule (Appendix D.3, Section 5.4) is designed to catch deterministically, independent of the ML models.

E.4 Windows Telemetry Modality (windows7 / windows10 datasets)

ts	CPU	MEM	DISK	PROCESS	THREADS	HANDLES	label
1558015400	8.2	525	1.2	explorer.exe	42	315	normal
1558015401	95.4	1400	88.3	powershell.exe	210	1402	attack
1558015402	12.7	610	3.6	svchost.exe	55	488	normal

E.5 Modality Summary

Modality	Observes	Typical Raw Features	Pipeline Function
Network	Communication between hosts	Protocol, ports, bytes, duration, connection state	preprocess_network
IoT	Physical device behavior	Sensor readings, device telemetry, actuator state	preprocess_iot_multi
Linux	Operating system activity	CPU, memory, disk I/O, process metrics, page faults	preprocess_linux_multi
Windows	Host resource behavior	CPU, RAM, disk, process/thread/handle counts	preprocess_windows

Appendix F: System Architecture Diagrams

These diagrams are generated directly from the pipeline structure described in Chapters 4 and 5, to give a visual complement to the textual architecture description.

F.1 High-Level System Architecture

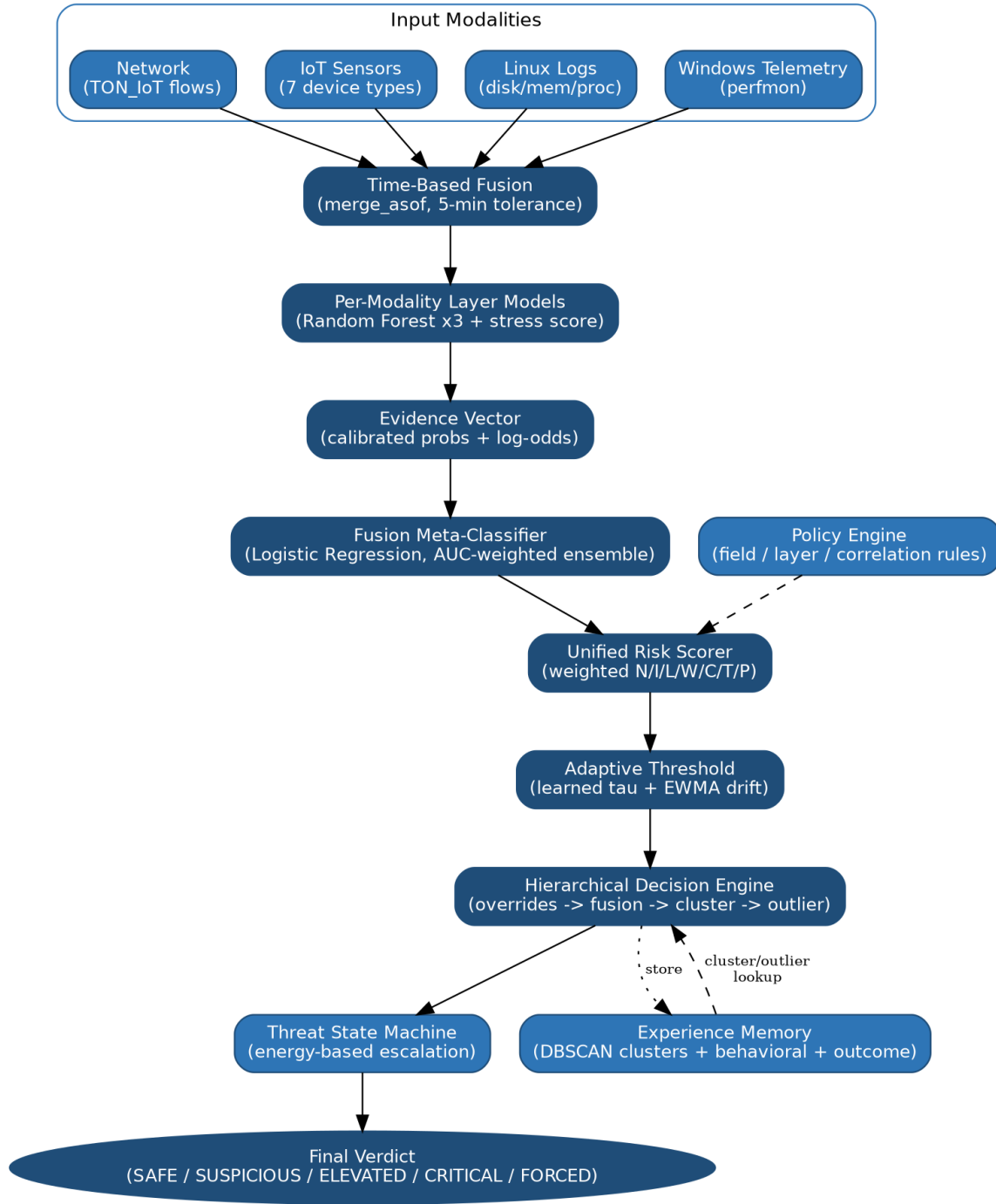


Figure F.1 — End-to-end CATF-IDS pipeline, from four input modalities through fusion, risk scoring, adaptive thresholding, and the threat state machine to a final verdict.

F.2 Hierarchical Decision Flow

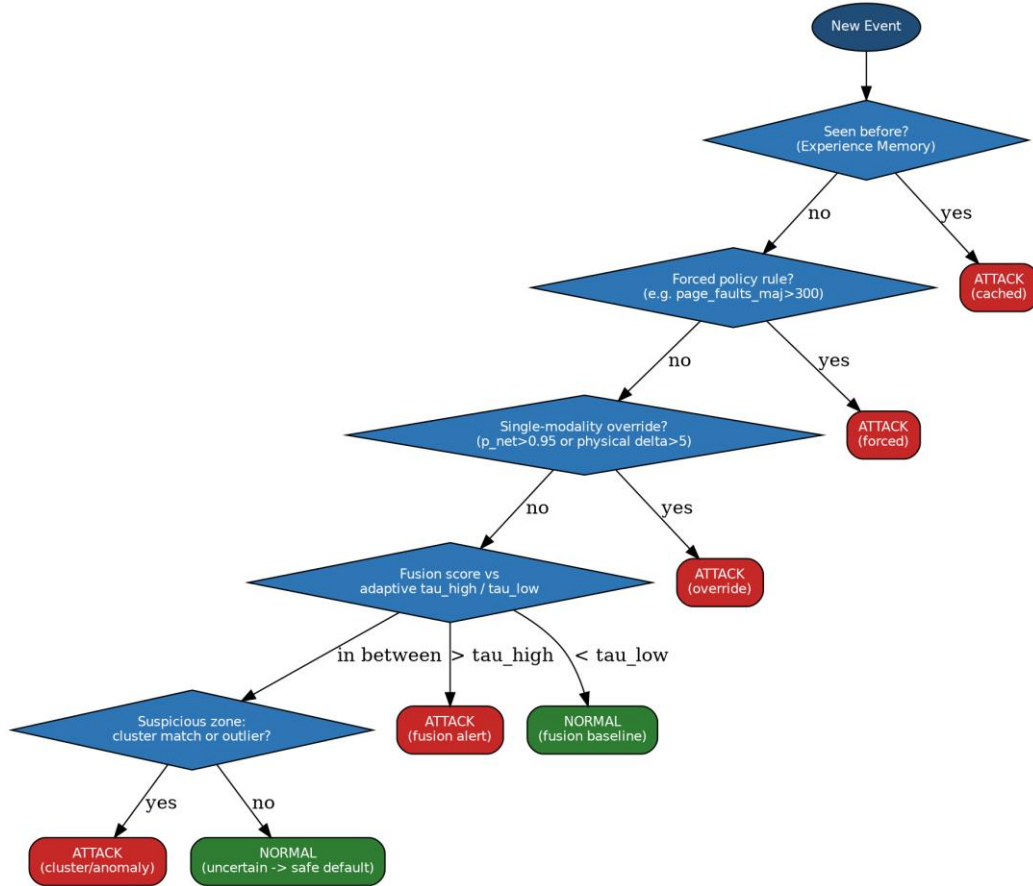


Figure F.2 — Order of evaluation inside `hierarchical_decision()`: cache lookup, forced policy rules, single-modality overrides, adaptive fusion threshold, then cluster/outlier resolution in the uncertain zone.

F.3 Threat State Machine

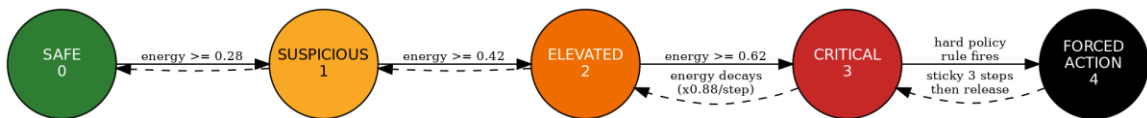


Figure F.3 — Five-level energy-based threat escalation. Energy decays each step (0.88 factor) unless sustained risk pushes it past the next threshold; `FORCED_ACTION` is sticky for 3 steps before releasing back to `CRITICAL`.